

Multi-Layer Perceptron

The three notebooks represent and demonstrate the fundamental concepts of a simple feedforward neural network (Multi-Layer Perceptron) implemented from scratch using the NumPy library. It specifically tackles the XOR problem, a classic benchmark for neural networks.

It specifically shows:

Activation Function: The definition and use of the sigmoid activation function and its derivative.

Network Architecture: Setting up input, hidden, and output layers.

Weights Initialization: How weights are initialized between layers.

Forward Propagation: How input data is passed through the network to generate predictions.

Error Calculation: How the difference between predicted and actual outputs (error) is calculated.

Backpropagation: The core algorithm for training neural networks, showing how errors are propagated backward through the network to adjust weights.

Weight Updates: How learning rate is used to modify weights based on the calculated deltas.

Training Loop: Running the forward and backward propagation steps over multiple epochs to minimize the error.

Performance Visualization: Plotting the error over epochs to observe the learning process.

Prediction: Using the trained weights to make predictions on new input data.

```
Compearing the outputs and the predictions

[67] 0 outputs
✓ 0 mp
array([[0],
       [1],
       [1],
       [0]])

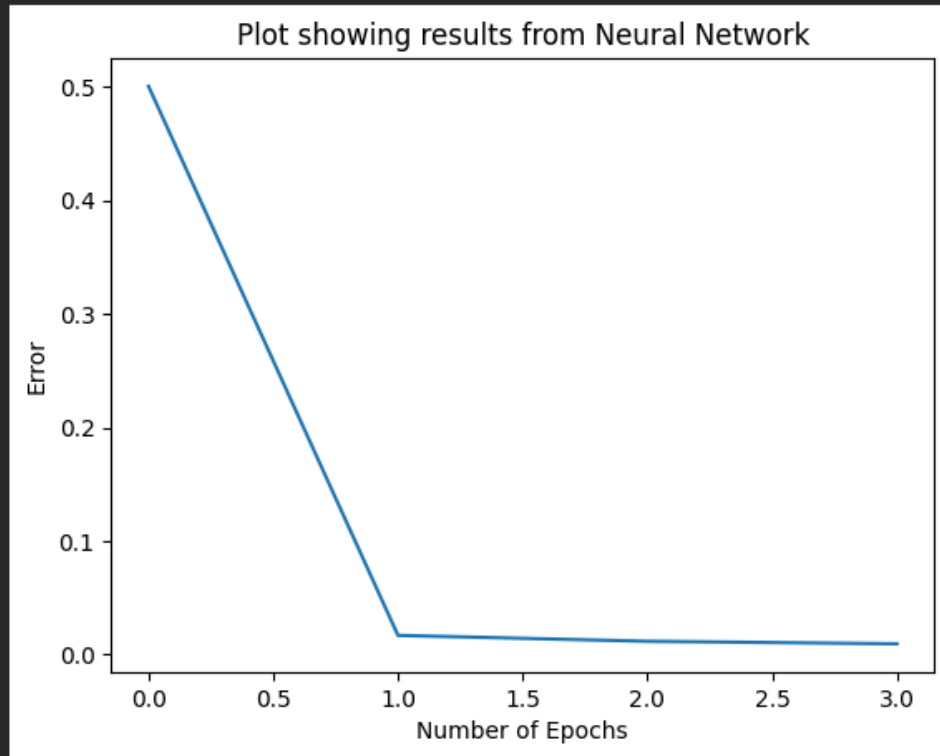
[68] 0 output_layer
✓ 0 mp
array([[0.01047024],
       [0.99213295],
       [0.99252728],
       [0.00727829]])

* We see that our neural network was able to get values close to the actual values from the results.
```

[66]
✓ 0
mp



```
1 plt.xlabel('Number of Epochs')  
2 plt.ylabel('Error')  
3 plt.title('Plot showing results from Neural Network')  
4 plt.plot(error)  
5 plt.show()
```



It is a really good walk-through of the workings of the above functions and methods. I'm only missing the emphasis on the role of bias (although it is implicitly covered).